

NDNSF: A Data-Centric Service Framework for Secure, Collaborative, and Dynamic Networked Applications

Tianxing Ma

Department of Computer Science
University of Memphis
Advisor: Dr. Lan Wang

PhD Proposal Defense
June 2026

Proposal Defense Roadmap

1. **Motivation and gap**
2. **NDNSF foundation and completed v0.1 evidence**
3. **Post-v0.1 collaboration plan and proposed enforcement**
4. **UAV and distributed-inference validation**
5. **Evaluation, risks, and Spring 2027 timeline**

NDNSF is the dissertation contribution; UAV and DI expose requirements and provide evidence.

Service Framework Problem

What a service framework must provide

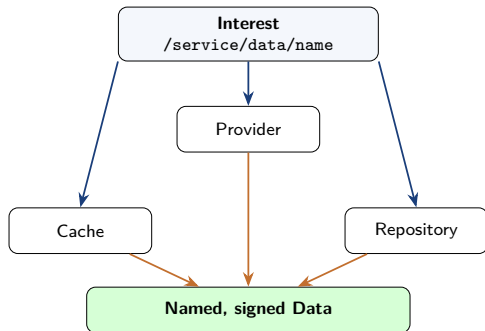
- ▶ Discover, authorize, invoke, and compose services
- ▶ Observe readiness and select Providers
- ▶ Manage retries, security, and data transfer

Friction in endpoint-oriented designs

- ▶ Calls bind to hosts, processes, brokers, or channels
- ▶ Discovery, identity, and trust sit outside invocation
- ▶ Stored or cached objects need separate protection

Dynamic multi-Provider applications still bridge endpoints, service policy, trust, and data by hand.

Why NDN for Services

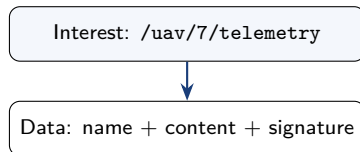


NDN supplies named retrieval and object authenticity; NDNSF defines the service and collaboration semantics above those primitives.

- ▶ NDN [1] lets consumers request **named Data**, not a host session.
- ▶ Data signatures remain verifiable across paths and storage.
- ▶ A Provider, cache, or repository can satisfy a matching Interest.
- ▶ Name-based retrieval reduces dependence on a fixed endpoint.

NDN Packets and Service Data

Interest	Name of desired Data plus optional selectors and parameters.
Data	Name, content, metadata, and producer signature.



Examples of service-relevant Named Data

- ▶ `/uav/7/telemetry/v=1042`
- ▶ `/uav/7/video/frame=918/seg=3`
- ▶ NDNSF Request, ACK, Selection, and Response
- ▶ model, tensor, and result artifacts

NDN [1] represents business inputs, service messages, and intermediate results as independently verifiable Named Data.

Supporting NDN Primitives

NDN Sync [2] / SVS [3]

Participants announce name and sequence-state changes so peers can fetch missing Data.

Data signatures [1]

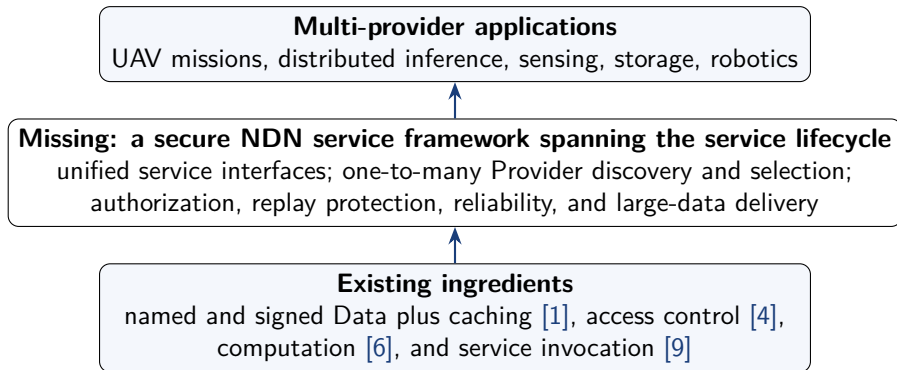
Consumers verify producer identity and object integrity whether Data is direct or cached.

NAC [4] / NAC-ABE [5,19]

Name-based encryption policy determines which identities can decrypt protected Data.

These primitives disseminate and protect individual objects; by themselves they do not assign Provider roles or authorize dependency edges in a multi-Provider execution.

Research Gap



NDNSF v0.1 supplies the core framework functions; the proposed dissertation work extends them to multi-role collaboration and verifies Data flow between assigned roles.

Related Work: NDN Service Framework Directions

Computation / composition: NFN [6], NFaaS [7], CFEC [8], CFN [14], CaSCON [15], and data-centric composition [21]

Named functions, placement, computation graphs, and dependency-aware composition.

Content authorization: NAC [4], NAC-ABE [5,19], Trust Schema [16], and access-controlled derived Data [20]

Protect content and constrain valid signers; policy is not derived from a runtime collaboration plan.

Invocation: RICE [9], NSC [10]

Named remote calls and execution; do not by themselves enforce a late-bound multi-role Data-admission policy.

Application-specific: MIA-NDN [11], DNMP [12], SECaaS [13]

Targets aggregation, measurement, or edge security.

Targeted collaboration gap. NDN v0.1 already covers secure invocation, runtime Provider discovery and selection, authorization, replay protection, and reliable Data delivery.

The proposed extension addresses a narrower missing function: after runtime ACKs, derive a multi-role dependency plan and admit intermediate Data only when its name, signer, assigned role, dependency edge, and request context match, including Data retrieved from an NDN cache.

NDNSF Contribution

Framework function	NDNSF contribution
Service interfaces and invocation v0.1	Generic C++ and Python APIs support Request, ACK, Selection, Response, and Targeted invocation without service-specific stubs; C++ also provides trusted same-process local calls.
One-to-many service model v0.1	One named Request reaches willing Providers; applications use built-in or customizable Provider-selection strategies.
Security and authorization v0.1	Name-based policy, NAC-ABE, signatures, one-time tokens, and replay checks protect invocation and Data.
Reliability and data delivery implemented	SVS retry, segmented retrieval, large-object references, streaming, and bounded FEC recovery.
Multi-role collaboration proposed work	Assign willing Providers to application roles, declare allowed Data flow, and verify Data received between assigned roles.

Research Questions

- RQ1 Which service-layer functions are required to support secure invocation, dynamic multi-Provider selection, authorization, and reliable Data delivery over NDN?
- RQ2 How can NDN-SF assign Providers to application roles and verify that each downstream Provider accepts Data only from the Provider assigned to an authorized upstream role?
- RQ3 What reliability, security, and latency tradeoffs arise from multi-Provider publication, selection, Targeted invocation, and bounded response retry or Provider reselection?
- RQ4 How should mobility and control requirements in UAV network applications be supported through NDN-SF?
- RQ5 How should distributed inference use NDN-SF to reuse model files, assign model parts to available devices at runtime, and run one model across multiple devices?

RQ2 tests role assignment and Data-flow verification for both directly received and cached Named Data.

Completed Foundation: NDNSF v0.1

- ▶ Implemented generic dynamic C++ API
- ▶ Multi-provider ACK collection
- ▶ Provider selection strategies
- ▶ NDN-SVS patch
- ▶ MiniNDN [22] evaluation

Provider API

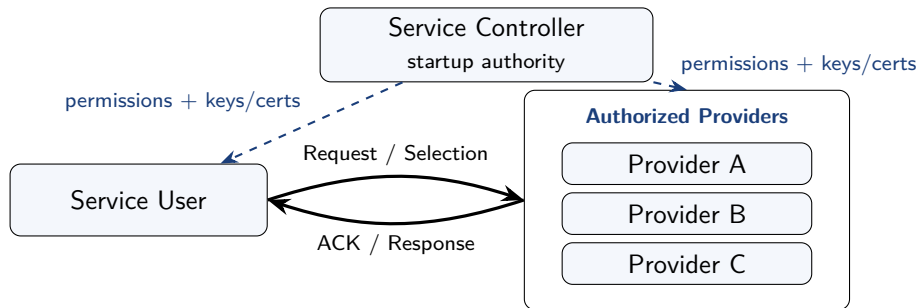
```
provider.addHandler<RequestT, ResponseT>(
    serviceName, handler);
```

User API

```
user.RequestService<RequestT, ResponseT>(
    providers, serviceName, request,
    onResponse, onTimeout, timeoutMs,
    strategy);
```

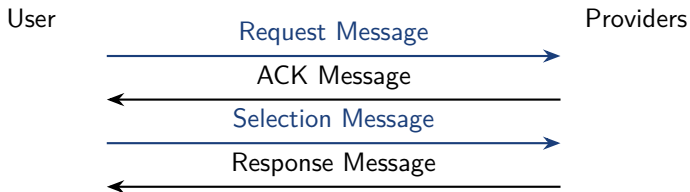
NDNSF v0.1 (commit be9421f..., tree c9cc0c...) is the May 20 repository baseline; role-based dependency collaboration is post-v0.1 work.

NDNSF Entity Roles



At startup, users and providers obtain permissions, keys, and certificates. During invocation, the user coordinates directly with providers through named Data messages.

NDNSF Invocation Workflow



Underlying NDN Sync [2]: announce, then fetch

1. Sender publishes each logical message as signed, named Data.
2. Sync announces the new Data name.
3. Receiver sends an Interest for that name and obtains the Data.

SVS [3] Pub/Sub API: combines the two-step delivery

- ▶ `publish(name, payload)` stores the Data and announces its name.
- ▶ `subscribe(prefix, callback)` learns the name, fetches the Data, then invokes the callback.

The NDNSF application does not manage the fetch itself.

ABE-Backed Authorization: Rationale and Limits

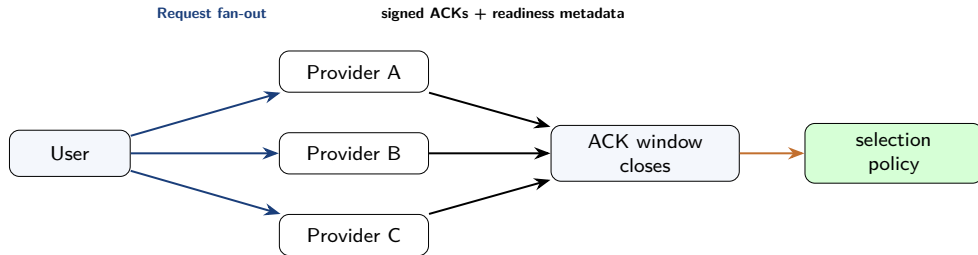
Why this design?

- ▶ NAC-ABE [19] protects discovery content for authorized Providers before selection.
- ▶ A DKEY aggregates service permissions; identity-signing credentials are reused.
- ▶ Runtime status checks and key rotation support revocation, with propagation and rekeying costs.

Message	Invocation binding
Request	fresh UserToken
ACK	echoed UserToken + fresh ProviderToken
Selection Response	selected Provider's ProviderToken echoed UserToken

Authorized but unselected Providers can read discovery content. DNMP [12] uses role keys; Trust Schema [16] authorization can also be paired with encryption.

Runtime Provider Discovery and Selection

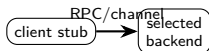


- ▶ Each Provider evaluates local readiness and may report load, wait time, capability, location, model availability, or safety state.
- ▶ After the ACK collection window closes, the User applies First Responding, Random, All Selected, or a custom policy.
- ▶ Selection authorizes the chosen Provider or Providers to execute.

Discovery asks who is willing and capable now; selection decides who should execute.

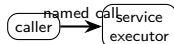
Baselines Before the Comparison

gRPC [17]



- ▶ Load balancing chooses among backend addresses supplied beforehand by a resolver; it does not discover willing Providers from an RPC.
- ▶ Here, gRPC-1 uses one endpoint; gRPC-SEQ-4 retries four preregistered endpoints sequentially.

NSC [10]



- ▶ Expresses an RPC-like service call with NDN names.
- ▶ The evaluated configuration binds one call to one executor.

NDNSF



- ▶ Discovers multiple ready Providers through one ACK window.
- ▶ Selects after observing runtime willingness and metadata.

Context only: NDNSF uses submitted v0.1 measurements; NSC/gRPC use one new run per loss with the shown workload and deadline. Batches and semantics differ; no variance or significance claim is made.

v0.1 Evidence: Network and Provider Failures

Contextual network-loss comparison



37 ms one-way MiniNDN route

- ▶ Memphis → CSU → UCLA; 5 ms service delay.
- ▶ 10 RPS; 5 s warmup; 60 s measured; 5 s deadline.

Success (%)	1%	3%	10%
NDNSF v0.1	99.83	99.67	89.83
NSC	90.17	74.00	34.67
gRPC	100.00	100.00	90.17

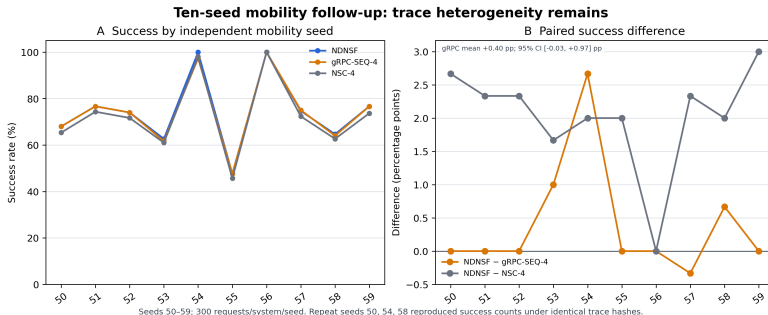
Intermittent Provider failures

- ▶ Providers reject or disappear with 10–30% probability.
- ▶ NDNSF selects among Providers that returned valid ACKs.

System	10%	20%	30%
NSC	83.25	66.58	49.92
gRPC	83.35	66.69	65.36
NDNSF	99.82	99.46	96.70

Context only: NDNSF uses submitted v0.1 results; NSC/gRPC use one new run per loss. Batches and Provider-selection semantics differ; no variance or significance claim is made.

Four-Provider Mobility Holdout



Follow-up: same one AP/four-Provider condition, 50 m/2 m/s, new seeds 50–59, same 4 s barrier, 60 s at 5 RPS, 1 s attempt/ACK, 5 s deadline, admission and health routing off.

Result: NDNSF 74.50%; gRPC-SEQ-4 74.10%; NSC-4 72.47%. NDNSF–gRPC is +0.40 pp, 95% CI [−0.03, +0.97] pp; the gate is not met.

What NDNSF v0.1 Still Lacks

Completed v0.1 evidence

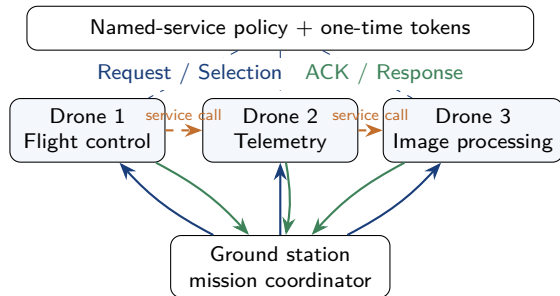
- ▶ Runtime discovery and service ACKs reporting willingness and status
- ▶ Provider selection under failure and mobility
- ▶ Request-level authorization and token binding

Still missing for collaboration

- ▶ Assign different Providers to different application roles, such as capture, detection, and mapping.
- ▶ Record which role is allowed to send Data to which downstream role.
- ▶ Verify that received Data came from the Provider assigned to the authorized preceding role, whether fetched directly or from a cache.

UAV and distributed inference are requirement generators: they expose why Provider selection alone is insufficient for multi-role execution.

UAV Workload and Requirements



- Mobility and changing reachability
- Command authorization
- Telemetry freshness
- Video and recording objects
- Multi-drone missions

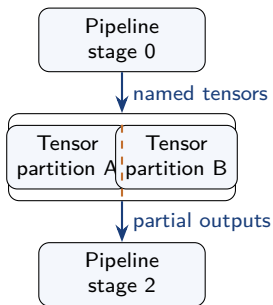
The ground station and drones may all invoke services; each drone can act as both an NDN SF User and Provider during a mission.

UAV Requirements → NDNSF Mechanisms

UAV requirement	NDNSF mechanism
Mobility and changing reachability	Service ACKs report willingness and runtime status; NDNSF selects a responding Provider. Optional bounded response reselection may run after selection.
Command authorization	Targeted invocation with NAC-ABE, one-time tokens, and replay checks
Telemetry freshness	Versioned names, freshness/expiry checks, and lifecycle state
Video and recording objects	Immutable object references, manifests, and segmented cache/Repo retrieval
Multi-drone missions	Assign Providers to mission roles; accept Data only from the Provider assigned to the authorized preceding role.

Generic one-to-many calls use ACK and Selection. Targeted calls skip them only after a Provider/service token batch has been bootstrapped.

Distributed Inference Motivation and Challenge



One run must align

- ▶ canonical model layers,
- ▶ pipeline stages and tensor partitions,
- ▶ Provider and device assignments,
- ▶ named dependencies and collectives.

Why it matters: Providers and visible devices are known only after ACK collection, yet every stage must assemble and consume the correct data.

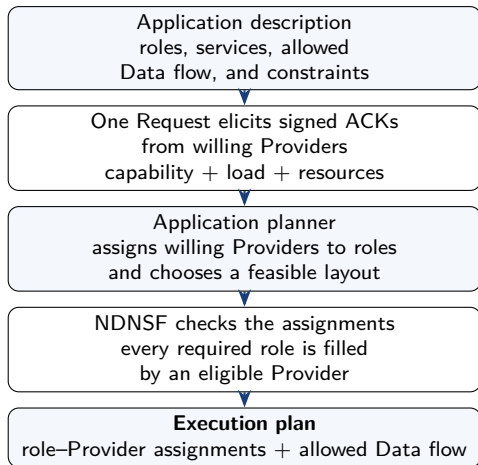
DI stresses the central problem: turn runtime Provider capabilities into a verified pipeline and tensor execution plan.

DI Requirements → Proposed NDNF-DI Mechanisms

Distributed-inference requirement	NDNF mechanism
Run a model that does not fit on one available device	Split the model into pipeline stages and, where supported, parallel tensor partitions
Use heterogeneous CPU and GPU Providers effectively	Collect signed Provider offers, then choose Providers and devices after the ACK window closes
Avoid copying a complete model for every request	Reuse signed layer artifacts from an NDN Repo or cache; each Provider prepares only its assigned model part
Send intermediate tensors only along the planned pipeline	Assign Providers to roles and verify each tensor's request, signer, preceding role, and name
Continue when a selected Provider becomes unavailable	Use authenticated status and bounded replanning or Provider reselection within the request deadline

NDNF carries Request, ACK, Selection, and Data. Proposed NDNF-DI mechanisms interpret the model, choose the split, assign devices, and run each model part.

Post-v0.1: From Application Roles to an Execution Plan



Example after ACK collection

Role	Provider
Capture	UAV-1
Detection	Edge-2
Mapping	GS-1

Allowed Data flow

- ▶ Capture image → Detection
- ▶ Detected objects → Mapping

Only Providers that returned a valid ACK for this request may be assigned a role.

Proposed: Verify Data Between Assigned Roles

YOLO example from the execution plan



Before P4 processes the tensor, it checks

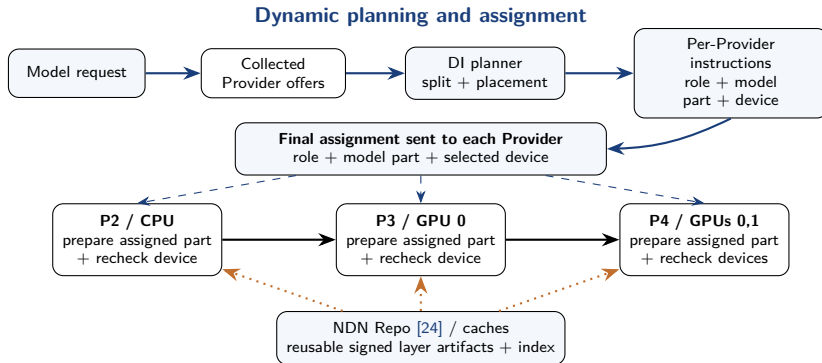
- ▶ it belongs to the current request;
- ▶ the signer is P2, assigned to Backbone;
- ▶ Backbone is the allowed preceding role;
- ▶ its name is under /tensor/features.

Expected decisions

- ▶ **Accept:** correctly signed P2 feature tensor.
- ▶ **Accept:** the same valid tensor retrieved from an NDN cache.
- ▶ **Reject:** unassigned P3 uses the same request ID.
- ▶ **Reject:** P2 publishes on an undeclared Data name.
- ▶ **Reject:** a valid old tensor is replayed for another request.

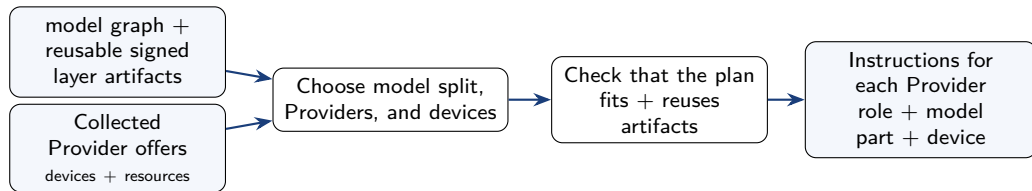
The proposed Plan-to-Data mechanism turns the execution plan into checks performed by each Provider.

Proposed NDNSF-DI Architecture



- ▶ NDNSF Core remains model-neutral; the proposed NDNSF-DI layer interprets the model, selects the split, and assigns Providers and devices.
- ▶ Providers prepare only their assigned model parts from reusable signed layer artifacts; the Repo does not store a new request-specific copy.
- ▶ A Provider offer reports current availability but does not reserve a GPU; each Provider rechecks resources immediately before loading.

Proposed Inference Plan Generation



Example decision

- ▶ stage 0 → P2 / CPU
- ▶ stage 1, partitions A–B → P4 / GPUs 0–1
- ▶ reuse signed layer artifacts already cached at P4

Resulting execution contract

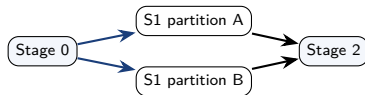
- ▶ Each Provider receives only its assigned role, model part, and device.
- ▶ Memory and topology are checked separately for every selected device.
- ▶ The same Data-flow checks apply to directly received and cached tensors.

Each signed Provider offer reports available devices, capacity, and cached signed layer artifacts.

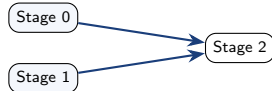
DI Hybrid Layout and Planner Decision

Candidate $N \times \{M_i\}$ layouts

Pipeline + tensor



Pipeline only



Planner compares

- ▶ model/device fit,
- ▶ activation/collective size,
- ▶ RTT, bandwidth, topology,
- ▶ per-device capacity and reuse.

Tradeoff

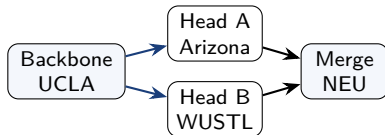
- ▶ Pipeline trades stage compute against activation transfer.
- ▶ Tensor sharding adds collective cost but can fit larger stages.

After timely ACKs are collected, the planner assigns each model part to one Provider and device; the Provider checks capacity again before execution.

Preliminary Diagnostic Experiment

Earlier role/chunk prototype on the same full-network MiniNDN harness, using a small-model ONNX [25] workload instrumented by NativeTracer; 10 successful real-ONNX runs per layout.

Shared-backbone layout



Measured end-to-end latency

Layout	Mean	p50	p95	SD
Shared backbone	259.4	267.7	283.9	22.1
Single Provider	186.8	191.7	206.3	20.0

All values are milliseconds.

Shared backbone uses four Providers; the single-Provider control runs all four roles at UCLA.

For this small model, the single-Provider layout is faster because it avoids fixed intermediate-data exchange. This diagnostic does not validate the proposed V3 reuse, hybrid partitioning, multi-GPU placement, or final resource recheck.

Planned DI Validation

- ▶ **Identity and reuse:** signed layer-artifact, assembled-fragment, and loaded-runtime identities remain distinct and reproducible.
- ▶ **Placement/admission:** CPU and 0/1/2+ visible accelerators; per-device feasibility; no ACK-time reservation; JIT revalidation.
- ▶ **Execution modes:** pipeline, adapter-certified tensor, and heterogeneous $N \times \{M_i\}$ hybrid cases.
- ▶ **Data/security:** dependency and collective edges, direct/cache equivalence, signer/name/key-scope negative tests.
- ▶ **Environments:** unit/integration harness first, then MiniNDN and sealed container/Tiger qualification.

Evidence remains gated: the V3 design is not presented as validated until model-file reuse, local assembly, device checks, and multi-device execution gates close.

Planned Evaluation Matrix

RQ	Dissertation Component	Evaluation Target	Evaluation Methodology
RQ1	Literature	Prior-work boundary	Literature and traceability analysis
RQ2– RQ3	Core	ACK-derived plan; role/edge admission; security, reliability, latency, overhead	Protocol tests; direct/cache tests; matched baselines
RQ4	UAV	Mobility; authorization; freshness; media; collaboration	MiniNDN/PX4 SITL [23] ; selected real machines
RQ5	DI	Reuse model files; assign model parts at runtime; assemble locally; run across devices	Integration tests; MiniNDN; sealed real-model/container tests

Rows state planned evidence; the earlier small-model campaign is preliminary diagnostic evidence.

Risk, Mitigation, and Timeline

Primary risks and controls

- ▶ **UAV scope:** validation workload.
- ▶ **Collaboration latency:** measure admission overhead.
- ▶ **DI data volume:** segment and measure transfers.
- ▶ **Variable hardware:** retain emulated baselines.

Spring 2027 completion path

- Aug. 2026 Plan-to-Data evaluation
- Sept. 2026 UAV role/edge validation
- Oct. 2026 Real-model DI/failure gates
- Nov.–Dec. 2026 Evidence freeze/chair draft
- Jan.–Mar. 2027 Review, defense, corrections, ProQuest

Defense by mid-Feb. 2027; ProQuest: Mar. 5 target / Mar. 26 deadline; Spring 2027 graduation.

Expected Contribution and Success Criteria

Mechanism

An ACK-derived collaboration plan assigns valid runtime candidates to roles and names every dependency edge.

Enforcement

A Provider accepts intermediate Data only for the current request, assigned producer role, declared edge, and permitted name/key scope.

Evidence

Direct/cache equivalence, same-request negative tests, matched baselines, and measured security, latency, and message overhead.

NDNSF is the dissertation contribution; UAV and distributed inference are validation workloads that test and refine it.

Dissertation thesis

NDNSF can turn ACK-derived roles and named dependencies into enforceable policy for direct and cached Named Data.

The v0.1 implementation and initial evidence exist; Plan-to-Data enforcement and full validation remain proposed.

Questions?

Backup: UAV Validation Details

- ▶ **Targeted command lifecycle:** latency, timeout reason, explicit retry.
- ▶ **Typed state freshness:** telemetry, mission, video, and safety state.
- ▶ **Large media objects:** video and recording as signed segmented Data.
- ▶ **Mission recovery:** cancellation, partial recovery, and bounded role reassignment.

UAV validation asks whether NDNFSF mechanisms support mobile, command-sensitive service use cases.

Backup: Runtime Metadata Evidence

► **Adaptive admission**

Configuration: open-loop 100 RPS offered load; compare no admission control with admission-aware NDNFSF.

Result: P95 latency stays at 268.12 ms with 0.00% timeout rate.

Meaning: providers should expose runtime capacity, not only service names.

► **Positive/negative ACK readiness**

Configuration: nearest provider is overloaded; the ACK-all baseline still selects it for 2209 of 2250 requests.

Result: success rises from 1.82% to 100.00% when negative ACKs exclude the overloaded provider.

Meaning: providers still return an ACK, but its status prevents an unavailable provider from becoming a selection candidate.

Backup: User-Side Admission Control

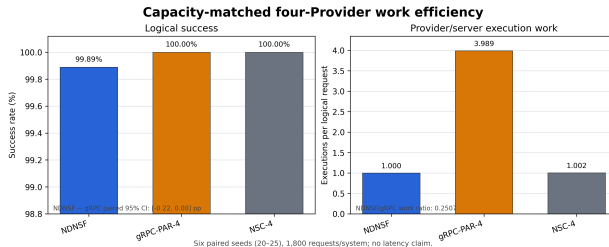
- ▶ NDNFSF bounds the number of outstanding service requests before they enter the Request–ACK–Selection–Response pipeline.
- ▶ Each request is locally admitted, queued, or rejected according to a window updated every 500 ms.
- ▶ The window grows when latency is stable, but shrinks under rising latency, timeouts, or near-full queues.
- ▶ Queue limits are derived from the current window, so overload protection adapts to runtime capacity instead of using one static threshold.

100 RPS offered load	Admission enabled	Admission disabled
P95 latency	268.12 ms at 81 actual RPS	4254.39 ms at 100 actual RPS
Timeout rate	0.00%	3.55%

Backup: NDNF Multi-Provider Coordination

The NDNF paper treats provider coordination as a framework concern.

- ▶ One Request reaches multiple authorized providers for one named service.
- ▶ ACK metadata reports readiness; the user selects after ACK collection.
- ▶ Selected providers execute and publish Responses under NDNF security.
- ▶ The mechanism is framework-level coordination, not application glue.



Capacity-matched, block-network gated: comparable success, 5,380 duplicate provider executions avoided. This supports work efficiency, not general mobility reliability or latency superiority.

Backup: Why Not ABE Plus Role-Certificate Authorization?

Both designs are valid; compare their permission-management costs.

- ▶ **Alternative:** ABE protects discovery; service-role certificates and Trust Schema rules authorize execution.
- ▶ **NDNSF:** challenges reuse attribute-based permission management, avoiding extra service-role signing credentials.
- ▶ **Additional cost:** User permission DKEYs and challenge processing are not already paid for by Provider-side discovery.
- ▶ Both designs still validate signatures, request identity, current authority, and replay state.

Fewer credential objects do not prove lower total cost. Compare issuance, updates, key sizes, and revocation overhead under the same permissions.

Backup: Middleware Comparison

gRPC [17] and ROS 2 [18] are useful, but they solve a different runtime problem.

- ▶ **gRPC:** strong RPC abstraction with name resolution, load balancing, and retry, but its channels carry messages between resolved service endpoints.
- ▶ **ROS 2:** strong robotics middleware built around a distributed robotics graph rather than NDN's named, cacheable Data model.
- ▶ **NDN advantage:** named, signed Data can be retrieved by name from providers, caches, or repositories.
- ▶ **NDNSF contribution:** adds authorization, ACK metadata, provider selection, replay protection, and large-data references above NDN.

NDNSF is not a replacement for every middleware stack; it studies what a data-centric service framework enables in dynamic, multi-provider environments.

References [1–7]

- [1] L. Zhang et al., “Named Data Networking,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 66–73, 2014.
- [2] T. Li et al., “A Brief Introduction to NDN Dataset Synchronization (NDN Sync),” in *Proc. IEEE MILCOM*, pp. 612–618, 2018.
- [3] P. Moll et al., “A Brief Introduction to State Vector Sync,” Named Data Networking Project, Tech. Rep. NDN-0073, rev. 2, 2021.
- [4] Z. Zhang et al., “NAC: Automating Access Control via Named Data,” in *Proc. IEEE MILCOM*, 2018.
- [5] C. A. Lee et al., “Supporting Virtual Organizations Using Attribute-Based Encryption in Named Data Networking,” in *Proc. IEEE CIC*, pp. 188–196, 2018.
- [6] C. Tschudin and M. Sifalakis, “Named Functions and Cached Computations,” in *Proc. IEEE CCNC*, pp. 851–857, 2014.
- [7] M. Krol and I. Psaras, “NFaaS: Named Function as a Service,” in *Proc. ACM ICN*, pp. 134–144, 2017.

References [8–13]

- [8] M. A. ur Rehman and B.-S. Kim, “CFEC: An Ultra-Low Latency Microservices-Based In-Network Computing Framework for Information-Centric IoVs,” *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3199–3212, 2024.
- [9] M. Krol et al., “RICE: Remote Method Invocation in ICN,” in *Proc. ACM ICN*, pp. 1–11, 2018.
- [10] D. Meirovitch and L. Zhang, “NSC—Named Service Calls, or a Remote Procedure Call for NDN,” Named Data Networking Project, Tech. Rep. NDN-0074, rev. 1, 2021.
- [11] M. Imran et al., “MIA-NDN: Microservice-Centric Interest Aggregation in Named Data Networking,” *Sensors*, vol. 23, no. 3, Art. no. 1411, 2023.
- [12] K. Nichols, “Lessons Learned Building a Secure Network Measurement Framework Using Basic NDN,” in *Proc. ACM ICN*, pp. 112–122, 2019.
- [13] R. Tourani et al., “Towards Security-as-a-Service in Multi-Access Edge,” in *Proc. ACM/IEEE SEC*, pp. 358–363, 2019.

References [14–21]

- [14] M. Król et al., “Compute First Networking: Distributed Computing Meets ICN,” *ACM ICN*, 2019.
- [15] C. Barrios et al., “CaSCON: Cache-Assisted Service Composition Orchestration Leveraging NDN,” *IEEE SMARTCOMP*, 2025.
- [16] Y. Yu et al., “Schematizing Trust in Named Data Networking,” *ACM ICN*, 2015.
- [17] gRPC Authors, “Core Concepts, Architecture and Lifecycle,” gRPC Documentation, accessed July 2026.
- [18] S. Macenski et al., “Robot Operating System 2: Design, Architecture, and Uses in the Wild,” *Science Robotics*, 2022.
- [19] S. Dulal et al., “Enhancing NAC-ABE to Support Access Control for mHealth Applications and Beyond,” *arXiv:2311.07299*, 2023.
- [20] C. Marxer et al., “Access-Controlled In-Network Processing of Named Data,” *ACM ICN*, 2016.
- [21] S. D. Fu et al., “Toward Data-Centric Service Composition,” *ACM HotNets*, 2024.

References [22–25]

- [22] Mini-NDN Developers, “Mini-NDN: A Mininet-Based NDN Emulator,” project documentation, accessed August 2026.
- [23] PX4 Development Team, “Simulation: Software in the Loop (SITL),” *PX4 User Guide*, accessed August 2026.
- [24] Named Data Networking Project, “Repo Protocol Specification,” repo-ng documentation, accessed August 2026.
- [25] ONNX Project, “Open Neural Network Exchange Intermediate Representation Specification,” ONNX documentation, accessed August 2026.